

Contents

1	Digital Toy 제작의 배경	3
1.1	Digital toy의 철학: 감각과 수학의 재결합	4
1.1.1	Definition	4
1.1.2	Previous works : Explorable Explanation Pioneering Artists	4
1.1.3	Digital Toy의 구현철학	5
1.1.4	Digital Toy의 구현방향	6
2	Digital Toy의 구현 방법	9
2.1	Digital Toy의 Framework	10
2.1.1	인터랙션 구조 : 어떤 구성요소로 세상을 구현할 것인가	10
2.2	기술 스택	11
2.2.1	개념 시각화 / 인터랙션	11
2.2.2	물리 시뮬레이션 / 수치계산	11
2.2.3	학습 플랫폼 / 웹 배포	11
2.2.4	메타적 방향	12
2.3	구현 로드맵	13
2.3.1	기술 난이도별 로드맵 — “디지털토이 기술 성장의 나선형 단계”	13
2.3.2	철학-기술 매핑표 — “생각을 구현하는 언어의 대응관계”	14
2.3.3	마무리 루프 — “Digital Toy Cycle”	15
3	인접 분야 : Scientific Visualization	17
3.1	한 문장 요약	18
3.2	문제의식과 정의	18

3.2.1	왜 이 분야인가	18
3.2.2	Scientific Visualization의 정의	18
3.3	분야 지도(맵)	19
3.3.1	대표 데이터/도메인	19
3.3.2	기술 계층	19
3.4	디지털 토이와의 관계	20
3.4.1	목적의 차이와 접점	20
3.4.2	디지털 토이 설계 포인트(필수)	20
3.5	학습 리소스(이론·강의)	21
3.5.1	교재/강의(개념)	21
3.5.2	참고 허브/기관	21
3.6	실습 리소스(파이썬·연구툴·웹)	22
3.6.1	Python 기반(입문 중급)	22
3.6.2	연구/산업 도구(중 고급)	22
3.6.3	웹 기반(경량·배포 용이)	22
3.7	웹에서의 고용량 FEM: 현실 해법	23
3.7.1	핵심 전략(결론)	23
3.7.2	가능/불가능 경계	23
3.7.3	예시/참고	23
3.8	자기주도 프로젝트 로드맵(예시)	24
3.9	색/인지 디자인 빠른 체크리스트	25
3.10	결론	25
3.11	부록 A: 빠른 링크 모음	25

Chapter 1

Digital Toy 제작의 배경

1.1 Digital toy의 철학: 감각과 수학의 재결합

1.1.1 Definition

Digital Toy = 감각으로는 접근할 수 없는 물리적·수학적 구조를 조작 가능한 형태로 구현하여, 인간의 직관과 개념 사이를 연결하는 교량.

1.1.2 Previous works : Explorable Explanation Pioneering Artists

STEM digital toys

1. Quantum Flytrap

<https://quantumflytrap.com/>

2. Edumedia

<https://www.edumedia.com/en>

Artists

1. Explorable Explanations Hub

- <https://explorabl.es/>

2. Nicky Case

- <https://ncase.me/>

- <https://blog.ncase.me/how-i-make-an-explorable-explanation/>

3. Amir Patel

- <https://www.redblobgames.com/>

4. Jack Schaedler

<https://jackschaedler.github.io/>

Bret Victor

5. Bret Victor의 ExplorableExplanations 철학 :

<https://worrydream.com/ExplorableExplanations/>

6. Combining digital and physical world : "Dynamic land" project

<https://dynamicland.org/2024/Intro/>

1.1.3 Digital Toy의 구현철학

(1) 감각 불가능한 세계를 '조작 가능한 경험'으로 전환

Digital Toy는 감각적 시뮬레이션을 통해 인간이 접근할 수 없는 추상적 세계를 조작 가능한 형태로 전환하며 탐험하는 기술적 인공감각이다.

(2) 수학적 이해의 역할 변화

수학은 시작점이 아니라 구조적 해석의 종착점이 되어야 한다.

기존 학습 구조	디지털 장난감 기반 학습
수학식 → 개념 → 현상	현상(시물) → 구조감각 → 수식
추상 → 구체	구체적 조작 → 추상적 인식
“증명”이 목적	“감각화된 이해”가 목적

(3) 철학적 배경

이건 단순한 교육공학이 아니라, 지식과 감각의 관계에 대한 철학적 실험이다.

철학적 대응	핵심 개념
피아제의 구성주의	지식은 조작과 상호작용 속에서 형성된다
메를로퐁티의 현상학	세계는 ‘지각적 행위’로 드러난다
파파트의 Constructionism	배우는 것은 ‘만들며 놀 때’ 일어난다
브렛 빅터의 “동적 매체”	수식과 직관을 연결하는 인터랙티브 매체의 가능성

(4) 디지털 토이의 구현 철학 요약

구분	설명	Young에게의 의미
탐험적 설명	사용자가 조작하면서 개념을 발견하게 하는 인터랙션 설계.	“보는 공부”가 아니라 “손으로 생각하는 공부.”
Toy not Tool	기능보다 놀잇감으로서의 경험. 결과보다 과정을 즐기게 함.	완결된 제품보다 “조작이 사유를 유도”하는 인터랙션.
Digital Humanism	기술로 인간의 사유 능력을 확장하되 대체하지 않음.	“AI 시대에도 손으로 배우는 인간”으로서의 학습윤리와 직결.

1.1.4 Digital Toy의 구현방향

(1) 기본 구현 방향

축	질문	구현 방향
감각화 (Perceptualization)	어떻게 ‘보이지 않는 것’을 느끼게 만들까?	시각·청각·촉각 시뮬레이션
구조화 (Formalization)	그 경험이 어떤 수학적 구조와 연결되는가?	미분방정식, 위상, 행렬구조
내면화 (Internalization)	사용자는 어떻게 ‘이해했다’는 느낌을 받는가?	인터랙션, 피드백, 게임성

(2) 은유 기반 디지털 토이: 공학 개념을 감각적 메타포로 바꾸기

1. 시각화를 넘어, 메타포적 이해로 기존의 *Explorable Explanation*들은 대부분 수식을 시각화하거나 변수를 조작하여 결과를 보여주는 수준에 머문다. 하지만 은유 기반 디지털 토이는 그 다음 단계를 향한다:

수식 $\Rightarrow$ 개념 $\Rightarrow$ 현상 $\Rightarrow$ 감각 $\Rightarrow$ 비유(analogy) $\Rightarrow$ 감각적 체험

이는 추상적 수학 기호를 감각적 은유로 변환하여, 학습자가 ‘이해했다’가 아니라 ‘느꼈다’고 말하게 만드는 시도이다.

2. 은유적 접근의 강점

- **감정과 직관의 결합** — 학습자가 머리로 이해하기보다 몸으로 느끼게 된다.
- **언어 이전의 사고 복원** — 수식을 배우기 전에 물리적 직관을 세우고, 이후 개념을 언어화한다.
- **문화적 확장성** — 도토리, 산, 물, 리듬 등 한국적 정서를 반영한 은유 가능.

3. “Analogy-driven Digital Toy”의 제작 단계

- (a) **메타포 스케치**: 개념이 무엇을 닮았는가를 정의 (예: 전류 = 물의 흐름, 위상 = 회전 리듬).

- (b) **감각 매핑**: 시각, 청각, 공간적 요소로 변환 (색 변화, 진동, 속도감 등).
- (c) **조작 인터페이스**: 사용자가 손으로 개념을 ‘만질 수 있게’ 디자인.
- (d) **해설 레이어**: 은유와 수식 사이의 다리 역할 (메타포 ↔ 방정식 대응표).

4. 개념적 은유에서 감각적 체험으로 : 예시

개념	기존 시각화	은유형 디지털 토이
Phasor	회전 벡터 그래프	원형 트랙 위를 달리는 주자. 각 속도 = 주파수, 주자의 그림자가 실수축에 투영되어 $\sin/\cos$ 신호가 생성됨.
선적분	$\int F \cdot dr$ 계산	사람이 길을 걸으며 도토리를 줍는 장면. 함수값을 ‘수확’으로 시각화, 총합이 적분값.
전위차	두 점의 전위차	산의 고도차 or 물줄기 낙차 표현, 전하가 흘러내리는 ‘지형’ 은유.
커패시터 충전	지수함수 그래프	병 속 물이 천천히 차오르되 속도가 점점 느려지는 리듬으로 표현.
자기유도	자속 변화 $\Rightarrow$ 전류	움직이는 금속고리에 ‘보이지 않는 손’이 작용. 자속이 변할 때 저항감이 생기는 감각적 인터랙션.
공진	진폭 주파수 그래프	그네 밀기 게임. 타이밍이 맞을 때만 에너지가 축적됨.
라플라시안	공간상의 2차 미분	풍선 표면에서 압력이 균일화되려는 ‘평탄화 본능’으로 표현.

5. 작은 시작 아이디어

- RLC 회로 리듬 시뮬레이터 — 커패시터는 ‘숨쉬기’, 인덕터는 ‘관성’, 저항은 ‘피로’.
- 푸리에 음악 상자 — 각 주파수 성분을 실제 소리와 파형으로 동시에 시각화.
- 전위 언덕 시뮬레이터 — 공이 굴러내리며 전류를 생성하는 낙차 기반 시각화.

Chapter 2

Digital Toy의 구현 방법

2.1 Digital Toy의 Framework

2.1.1 인터랙션 구조 : 어떤 구성요소로 세상을 구현할 것인가

모듈 개념	설명	구현 포인트
Scene / Object / Interaction	물리 세계의 최소 단위: 장(scene), 물체(object), 사용자 조작(inter-action).	p5.js나 Three.js에서 기본 단위로 삼기 좋음.
State ↔ Visualization	수학적 상태를 화면에 실시간으로 반영.	상태(state)와 표현(render)을 분리. 물리엔진처럼 확장 가능.
Parameter Flow	슬라이더나 입력값이 파라미터 → 계산 → 시각화로 흘러감.	“수학이 음악처럼 연주되는” 감각적 흐름 설계.
Narrative Layer	단순히 보여주는 것이 아니라 변화의 이유를 느끼게 하는 시각적 리듬.	변화가 ‘생각의 리듬’을 만든다. 예: 점이 가속될 때 색상이 변한다.

2.2 기술 스택

2.2.1 개념 시각화 / 인터랙션

도구	특징	추천 상황
p5.js	Processing의 웹버전. 가장 직관적이고 예술적.	아이디어 시각화, 수학·물리 실험용.
Three.js	3D 장면 구현, 광학·산란계 시각화에도 응용 가능.	광학 장, 입자 산란 등 3D 효과 필요 시.
D3.js	데이터 기반 시각화. 함수 그래프나 파형 실험에 적합.	Fourier, 회로 응답 등 시그널 기반 실험용.

2.2.2 물리 시뮬레이션 / 수치계산

도구	특징	추천 상황
NumPy + Matplotlib (Python)	수학모델 실험용. 결과를 JS로 이식 가능.	개념 검증, 산란/파동 전처리 계산.
PyScript	Python을 HTML 안에서 바로 실행.	p5.js에 Python 로직을 혼합해 하이브리드 구현 가능.
GPU.js / WebGL	브라우저에서 병렬 계산.	Light propagation, interference 등.

2.2.3 학습 플랫폼 / 웹 배포

도구	특징	추천 상황
Jekyll(GitHub)	정적 사이트로 손쉬운 포트폴리오 구축.	이미 사용 중이므로 토이 갤러리화 가능.
ObservableHQ	JS 기반 실험 공유 플랫폼.	프로토타입 공유, 수학 실험 기록용.
Next.js + p5-wrapper	본격적 인터랙티브 전시 사이트 구축.	“디지털 토이 뮤지엄”으로 발전 가능.

2.2.4 메타적 방향

“코딩 공부”가 아니라 “사고의 인터페이스 디자인”이다.

- 목표: “어떤 사고과정을 시각화하고 싶은가?”를 먼저 정의.
- 표현: 수식 → 상태 → 시각 → 감각 의 단계적 전환.
- 루프: 코드 수정 → 즉시 피드백 → 개념 변화.

이 루프를 반복하면서 “손으로 사고하는 인간”으로서의 감각이 회복된다. AI 시대의 디지털 토이는 결국 사유의 촉각화 장치이다.

2.3 구현 로드맵

2.3.1 기술 난이도별 로드맵 — “디지털토이 기술 성장의 나선형 단계”

단계	목표	핵심 도구	학습·구현 포인트
Lv.0	Seed Toy (개념 발화기)	p5.js	점, 선, 마우스 인터랙션으로 기본 함수나 물리현상 표현. <code>setup()</code> - <code>draw()</code> 구조 익히기.
Lv.1	Modular Toy (조작 가능한 세계)	p5.js + GUI	슬라이더·토글로 변수 조작. State 관리(객체 기반).
Lv.2	Analytic Toy (계산 가능한 세계)	Python (NumPy, Matplotlib) → JS 이식	수치통합, FFT, 필터, 확률 분포 등 모델 실험. 이후 JS로 시각화.
Lv.3	Spatial Toy (공간적 시각화)	Three.js / WebGL	카메라, 광원, 셰이더 이해. 장(field)의 깊이감 표현.
Lv.4	Hybrid Toy (언어 융합형)	PyScript, GPU.js	브라우저 내 수치계산 + 시각화 결합. GPU 연산.
Lv.5	Narrative System (철학적 전시형)	Jekyll / ObservableHQ / Next.js	디지털토이들을 연결해 내러티브 구성. 데이터 기록·설명 자동화.
Lv.6	Toy Engine (창작 프레임워크)	Custom JS Framework	Scene, Object, State, UI를 모듈화. p5.js의 상위 추상화 설계.

요약하자면:

p5.js → Python(NumPy) → Three.js → WebGL → Web Framework

이것은 단순한 도구의 나열이 아니라, “수학적 사고의 실시간 시각화 능력”이 확장되는 궤적이다.

2.3.2 철학-기술 매핑표 — “생각을 구현하는 언어의 대응관계”

철학적 개념	대응 기술 개념	구현 방식	구현 시 사유 포인트
발견 가능한 설명	p5.js + GUI	슬라이더 · 토글로 직접 조작 하며 수학 · 물리식 체험	“설명”이 아니라 “탐구 경험”이 목표.
Toy not Tool	Minimal UI / Natural interaction	결과보다 조작감과 피드백 중심	“정답”보다 “움직임이 주는 의미”를 설계.
Digital Humanism	브라우저 기반 공개형 구조 (GitHub Pages, Observable)	누구나 접근 · 수정 가능한 오픈 플랫폼	기술이 인간의 이해를 돕는 지 자문하라.
State ↔ Visualization	Reactivity (draw-loop, data binding)	내부 상태를 즉시 화면에 반영	눈으로 ‘계산의 리듬’을 느끼게 한다.
Narrative Toy (사유의 여정의 로서의 토이)	Story-driven visualization / Animated transitions	변수 변화에 의미 부여. 시간 · 리듬 활용	“변화의 이유를 느끼게 하는” 흐름 설계.
Embodied Thinking (몸으로 사유하기)	마우스 · 터치 · 센서 입력	입력 제스처로 수식 제어	손의 움직임 = 사고의 움직임.
Analog Recovery (디지털로 감각 복원)	Noise / Imperfection / Hand-drawn aesthetic	픽셀, 흔들림, 불완전함을 의도적으로 표현	디지털의 냉정함 속에 인간의 질감을 남긴다.
Meta-Reflection	Data logging / Replay system	사용자의 조작 과정을 기록 · 재생	‘사유의 흔적’을 다시 보는 도구로 확장.

2.3.3 마무리 루프 — “Digital Toy Cycle”

Idea (철학적 질문)



Toy Prototype (p5.js)



Mathematical Model (Python)



Spatialization (Three.js / WebGL)



Narrative Integration (Jekyll / Observable)



Reflection & Rebuild

이 루프를 매번 한 바퀴 돌 때마다 → “철학이 기술이 되고, 기술이 철학이 되는” 감각이 쌓인다. 결국 디지털토이는 나 자신을 시각화하는 사유 장치가 된다.

Chapter 3

인접 분야 : Scientific Visualization

Scientific Visualization과 디지털 토이:

철학, 기술 스택, 학습 리소스, 그리고 웹 구현 전략 종합

3.1 한 문장 요약

Scientific Visualization(SciVis)은 복잡한 과학 데이터를 인간의 인지·감각 체계가 이해할 수 있는 형태로 번역하는 학문이다. 본 문서는 SciVis의 핵심 개념과 디지털 토이(Explorable Physics/Engineering Toys)에 즉시 활용 가능한 기술·학습 자원을 정리하고, 고용량 FEM의 웹 시각화를 위한 *Precompute + WebGPU/WebAssembly* 전략을 제시한다.

3.2 문제의식과 정의

3.2.1 왜 이 분야인가

- 전기·전자·광학·유체 등 감각으로 접근하기 어려운 현상을 학습/탐구하려면, 조작 가능한 시각적 인터페이스가 필요하다.
- 디지털 토이는 “개념 ↔ 직관”의 다리를 놓고, SciVis는 그 다리를 정교하게 설계하는 기술/이론을 제공한다.

3.2.2 Scientific Visualization의 정의

SciVis = 과학/공학/의학 데이터(필드, 메쉬, 볼륨, 시뮬레이션 결과)를 등치선/등면, 스트림라인, 볼륨 렌더링, 벡터장 표현 등으로 탐색 가능하게 시각화하는 분야.

(참고 구분) InfoVis: 추상/통계/네트워크 데이터 중심. SciVis: 물리적 연속장/시뮬레이션 데이터 중심.

3.3 분야 지도(맵)

3.3.1 대표 데이터/도메인

영역	예시 데이터	시각화 목표
유체역학	CFD: 속도/압력/와도 필드	유동 구조, 소용돌이, 충격파
전자기학	E/B 필드, 전위, Poynting 벡터	필드라인, 등전위면, 위상/진폭
전기기기	자속 밀도, 토크-슬립 곡선	자속선 애니메이션, 운영점 탐색
의료영상	MRI/DTI/CT 볼륨	조직 경계, 확산 경로
지구과학	온도/밀도/응력 3D 필드	단면/등면/시간 변화
광/양자	스펙트럼, 파동함수	복소 위상 색상 매핑, 간섭 패턴

3.3.2 기술 계층

계층	핵심 기술
데이터 처리	샘플링, 보간, 등치선/등면 추출, 메쉬/볼륨 변환
그래픽스	GPU 렌더링, Volume Rendering , Ray Casting, 셰이더
인터랙션	카메라/슬라이더/시간축, 피킹, 브러싱
인지 디자인	컬러맵(지각 균일성), 투명/명암, 모션 큐

3.4 디지털 토이와의 관계

3.4.1 목적의 차이와 접점

관점	Scientific Visualization	디지털 토이(Explorable)
목적	연구/분석 정확도	개념 직관 · 탐구 경험
사용자	연구자/엔지니어	학습자/일반인
도구	C++/VTK/ParaView/HPC	p5.js/Three.js/Web
핵심 가치	정량성 · 대규모 데이터	직관 · 피드백 · 게임성
접점	“보이지 않는 것을 보이게”	동일 철학, 경량 구현

3.4.2 디지털 토이 설계 포인트(필수)

- **Perceptual Mapping**: 물리량→색/크기/속도/음향.
- **Interactive Feedback**: 조작→즉시 반응(*Direct Manipulation*).
- **Simplification for Intuition**: 구조적 패턴을 먼저, 수식은 나중.
- **Dynamic Abstraction**: 동일 현상을 여러 추상화 높이에서 탐색(Bret Victor의 *Ladder of Abstraction*).

3.5 학습 리소스(이론 · 강의)

3.5.1 교재/강의(개념)

- The Visualization Toolkit (VTK) Book — Schroeder, Martin, Lorensen. <https://docs.vtk.org/>
- Visualization Analysis and Design — Tamara Munzner.
- Data Visualization: Principles and Practice — Hansen & Johnson.
- ETH Zürich Scientific Visualization 강의노트: <https://cgl.ethz.ch/teaching/scivis22/>
- Univ. of Utah CS 6630 Scientific Visualization: <https://sci.utah.edu/~beiwang/teaching/cs6630/>

3.5.2 참고 허브/기관

- IEEE SciVis Contest Archive: <https://sciviscontest.ieeevis.org/>
- SCI Institute (Univ. of Utah): <https://sci.utah.edu/>
- TU Wien VisGroup: <https://www.cg.tuwien.ac.at/research/vis/>

3.6 실습 리소스(파이썬 · 연구툴 · 웹)

3.6.1 Python 기반(입문 중급)

- PyVista (VTK 파이썬 래퍼): <https://docs.pyvista.org/>, 튜토리얼 <https://tutorial.pyvista.org/>
- Mayavi: <https://docs.entthought.com/mayavi/mayavi/>
- Matplotlib/Plotly 3D: 간단한 등치선/필드 화살표/표면 플롯에 적합.

3.6.2 연구/산업 도구(중 고급)

- VTK: <https://docs.vtk.org/> (C++/Python API 예제 풍부)
- ParaView: <https://www.paraview.org/tutorials/>
- VisIt: <https://wci.llnl.gov/simulation/computer-codes/visit>
- Blender SciVis(예술형): 유동/VDB импорт, 프레임 고품질 렌더.

3.6.3 웹 기반(경량 · 배포 용이)

- Three.js 매뉴얼: <https://threejs.org/manual/>
- WebGL Fundamentals: <https://webglfundamentals.org/>
- VTK.js: <https://kitware.github.io/vtk-js/>
- p5.js(교육 · 2D/간단한 3D): <https://p5js.org/learn/>
- Observable/D3(데이터형 인터랙션): <https://observablehq.com/>

3.7 웹에서의 고용량 FEM: 현실 해법

3.7.1 핵심 전략(결론)

브라우저에서도 FEM은 가능하나, 고용량 실시간 전계산은 어려움.

따라서 계산(FEM Solver)과 시각화(Viewer)를 분리:

1. 오프라인 사전 계산: FEniCS/deal.II/COMSOL/Ansys 등에서 해석 $\Rightarrow$.vtk/.xdmf/.vtu/.json 로 익스포트
2. 데이터 경량화: decimation, level-of-detail, 타임스텝 샘플링
3. 웹 렌더링: Three.js 또는 VTK.js + WebGL/WebGPU
4. 선택적 경량 재계산: WebAssembly(wasm)로 소형 solver 탑재(2D/소규모)

3.7.2 가능/불가능 경계

항목	현실성
2D FEM(수백 천여 노드) 실시간	가능 (wasm + 간단 solver)
3D 소형(수천 수만 요소) 인터랙션	부분 가능 (사전계산 + 효율적 전송)
3D 대규모(수십만 수백만 요소) 실시간	어려움 (서버 계산 + 스트리밍/프록시 필요)
전문 표현(등면/응력텐서/모드형상)	가능 (VTK.js/Three.js로 구현)

3.7.3 예시/참고

- VTK.js로 .vtk/.vtu 뷰어 제작: 등치면, 변형, 벡터장.
- FEniCS & Pyodide 실험: 브라우저 내 파이썬(wasm) 기반 미니 solver.
- WebGPU 데모(향후성): FEM/유체의 GPU 병렬 업데이트 프로토타입 다수.

3.8 자기주도 프로젝트 로드맵(예시)

월	목표	산출물
1	SciVis 개론 + PyVista 시작	등치선/등면, 스트림라인, 표면 플롯 실습 노트
2	볼륨/벡터 데이터 처리	전위/자속/속도장 시각화 샘플 2개
3	ParaView 워크플로우	외부 FEM/CFD 결과 импорт → 필터 파이프라인 저장
4	Three.js/VTK.js 전환	웹 뷰어 프로토타입(메쉬+스칼라/벡터 필드)
5	인터랙션/최적화	슬라이더 · 타임라인 · 픽킹, 데이터 다운샘플링
6	완성 프로젝트	Explorable FEM/EM 장난감 1개(학습 시나리오 포함)

3.9 색/인지 디자인 빠른 체크리스트

- 컬러맵: Perceptually Uniform(예: Viridis). ColorBrewer: <https://colorbrewer2.org/>
- 투명도: 깊이 단서 과도하면 혼탁 → 얇은 슬라이스/등면 병행.
- 모션 큐: 미세한 진동/흐름 애니메이션은 구조 파악에 유효.
- 라벨링: 단위/스케일바/레전드 최소한 표기.

3.10 결론

핵심: 디지털 토이의 목표는 정확도 경쟁이 아니라 직관의 다리 놓기.

SciVis의 철학과 기술 일부를 빌려와, 경량 인터랙션으로 조작 가능한 학습 경험을 제공하라.

고용량 FEM은 사전 계산 + 웹 시각화로, 중소형 문제는 *wasm/WebGPU*로 점진 확장하라.

3.11 부록 A: 빠른 링크 모음

- VTK: <https://docs.vtk.org/> — ParaView: <https://www.paraview.org/tutorials/>
- PyVista: <https://docs.pyvista.org/> — 튜토리얼: <https://tutorial.pyvista.org/>
- VTK.js: <https://kitware.github.io/vtk-js/> — Three.js: <https://threejs.org/manual/>
- WebGL Fundamentals: <https://webglfundamentals.org/> — p5.js: <https://p5js.org/learn/>
- IEEE SciVis Contest: <https://sciviscontest.ieeevis.org/>